Reviewer Pack — Cubical Betti Sum of Implicant Complex Lower-Bounds DNF-MCSP

Entry 420946ef183b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 12:47:16 UTC

Cubical Betti Sum of Implicant Complex Lower-Bounds DNF-MCSP

Entry ID: `420946ef183b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 12:47:16 UTC

1. Conjecture

Field A (mathematical branch): Cubical algebraic topology — total Betti number B(f)=Σ_k β_k(C_f;GF(2)) of the cubical subcomplex C_f of [0,1]^n whose k-cells are exactly the k-dimensional sub-cubes of the Boolean hypercube whose 2^k vertices all lie in supp(f); homology computed via GF(2) Smith Normal Form on the explicit cubical boundary operator (rarely deployed in circuit complexity; distinct from simplicial/persistent TDA used in data analysis) **Field B** (complexity object): DNF-MCSP: the meta-complexity problem of minimizing the number of product terms DNF_min(f) in a DNF representation of a Boolean function given its 2^n-bit truth table (Hirahara–Oliveira–Santhanam style restricted-MCSP variant)

Statement:

> For every non-constant Boolean function f:{0,1}^n→{0,1}, let C_f be the cubical implicant complex (k-cells = sub-cubes wholly contained in supp(f)) and let B(f)=Σ_{k=0}^{n} β_k(C_f;GF(2)). Conjecture: log_2(DNF_min(f)) ≤ B(f) for every non-constant f on n inputs, with the bound tight (up to additive 1) on parity-like functions and on disjoint-cube indicators. Equivalently, any DNF with fewer than 2^{B(f)} terms fails to compute f, providing a topological lower bound on DNF-MCSP.

Rationale (proposer's reasoning):

> Maximal cells of C_f are precisely the prime implicants, so a DNF cover is essentially a 0-cell-cover witnessing nerve-lemma collapse of C_f; any topological obstruction (β_0 components, β_1 cycles bridging clusters, higher voids) forces additional terms beyond the naive component count. While β_0 alone has been touched on tangentially, the *full* GF(2) cubical Betti sum has not been linked to DNF size because cubical (vs simplicial) homology is unfashionable and rarely computed on truth tables. If true, cubical homology yields a barrier-respecting lower-bound technique on DNF-MCSP and a fresh handle on Hirahara-style hardness magnification.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `27646652b756baaf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all 65536 truth tables for n=4 and 2000 uniform-random truth tables each for n=5 and n=6 (5 seeds), the inequality ⌈log_2(DNF_min(f))⌉ ≤ B(f) must hold for ≥99% of non-constant f per seed, with mean slack B(f)−⌈log_2 DNF_min(f)⌉ ≥ 0; total runtime <60s.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `cubical complex Boolean function Betti DNF lower bound` - `implicant complex homology monotone circuit complexity` - `topological lower bound DNF minimization Boolean hypercube`

Top relevant hits considered: - [http://arxiv.org/abs/1501.01331v1] DNF complexity of complete boolean functions - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2110.02458v3] Magnitude homology of graphs and discrete Morse theory on Asao-Izumihara complexes - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1805.08177v4] A Polynomial Time Delta-Decomposition Algorithm for Positive DNFs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.3s

5.1 Generated Python source

```python
import sys
import random
import math

def gaussian_elimination(A):
    n = len(A)
    m = len(A[0])
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
```

```
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                return None  # Singular matrix
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(m):
                    A[j][k] -= factor * A[i][k]
        rank = sum(1 for row in A if any(row))
        return rank

    def boundary_matrix(f, n):
        m = 2**n
        B = [[0]*m for _ in range(n)]
        for i in range(m):
            mask = i
            while mask:
                B[bin(mask).count('1')-1][i] = f[i]
                mask &= (mask - 1)
        return B

    def betti_numbers(B, n):
        betti = [0]*n
        for k in range(n):
            ker_dim = gaussian_elimination(B[:k+1])
            if ker_dim is None:
                return None  # Singular matrix
            rank = sum(1 for row in B[k] if any(row))
            betti[k] = ker_dim - rank
        return betti

    def dnf_min(f, n):
        m = 2**n
        prime_implicants = []
        for i in range(m):
            mask = i
            while mask:
                if all(f[j] == f[i] for j in range(m) if (j & mask) == mask):
                    prime_implicants.append(mask)
                mask &= (mask - 1)
        cover = set()
        for implicant in prime_implicants:
            cover |= {i for i in range(m) if implicant & i == implicant}
        return len(cover)

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        n_values = [5, 8, 11, 14]
        total_metric_value = 0
        instances_tested = 0
        conjecture_holds = True
        counterexample = ""

        for n in n_values:
            for _ in range(20):
                f = ''.join(str(random.randint(0, 1)) for _ in range(2**n))
                if f.count('1') == 0 or f.count('1') == 2**n:
```

```
                continue
            B = boundary_matrix(f, n)
            betti = betti_numbers(B, n)
            if betti is None:
                conjecture_holds = False
                counterexample = "singular_matrix"
                break
            DNF_min_val = dnf_min(f, n)
            metric_value = math.ceil(math.log2(DNF_min_val)) - sum(betti)
            total_metric_value += metric_value
            instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0
    support_fraction = instances_tested / (len(n_values) * 20)

    return {
        "metric_name": "Betti Sum - DNF Min",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fractio
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'Betti Sum - DNF Min', 'metric_value': 0, 'instances_tested': 0, 'conjecture_h
TRIAL: {'metric_name': 'Betti Sum - DNF Min', 'metric_value': 0, 'instances_tested': 0, 'conjecture_h
TRIAL: {'metric_name': 'Betti Sum - DNF Min', 'metric_value': 0, 'instances_tested': 0, 'conjecture_h
TRIAL: {'metric_name': 'Betti Sum - DNF Min', 'metric_value': 0, 'instances_tested': 0, 'conjecture_h
TRIAL: {'metric_name': 'Betti Sum - DNF Min', 'metric_value': 0, 'instances_tested': 0, 'conjecture_h
RESULT: FALSIFIED counterexample="singular_matrix" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The stdout shows the run actually FALSIFIED the conjecture (RESULT: FALSIFIED, counterexample='singular_matrix'), with instances_tested=0 across all trials — meaning the harness never successfully evaluated a single instance. This is a construction/implementation failure (likely a linear-algebra routine over GF(2) hitting a singular matrix during Betti computation), not genuine empirical support. There is no evidence here for SUPPORTED; the test never measured B(f) vs log2(DNF_min(f)) on any act

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test reported FALSIFIED with counterexample='singular_matrix' but instances_tested=0 across all trials, indicating a Betti-computation implementation failure (singular GF(2) matrix) rather than a genuine functional counterexample. No actual f was evaluated, so neither support nor falsification of the conjecture is demonstrated. | next: Fix the GF(2) boundary-rank computation (e.g., use a robust Smith normal form or sparse rank routine) so B(f) can be evaluated on real instances, then re-run

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	24572
2	propose	claude_max	opus	0	0	176375
3	preregistration	claude_max	opus	0	0	6672
4	novelty	claude_max	opus	0	0	5425
5	novelty	claude_max	opus	0	0	13252
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16695
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11941
8	critic	claude_max	opus	0	0	11251
9	judge	claude_max	opus	0	0	6011

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 272195 ms total latency. Provider mix: {'claude_max': 7, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/420946ef183b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/420946ef183b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/420946ef183b.tar.gz` (if generated)